PhishGuard

An AI Phishing-Detection Mail Agent

Project Plan & Build Roadmap

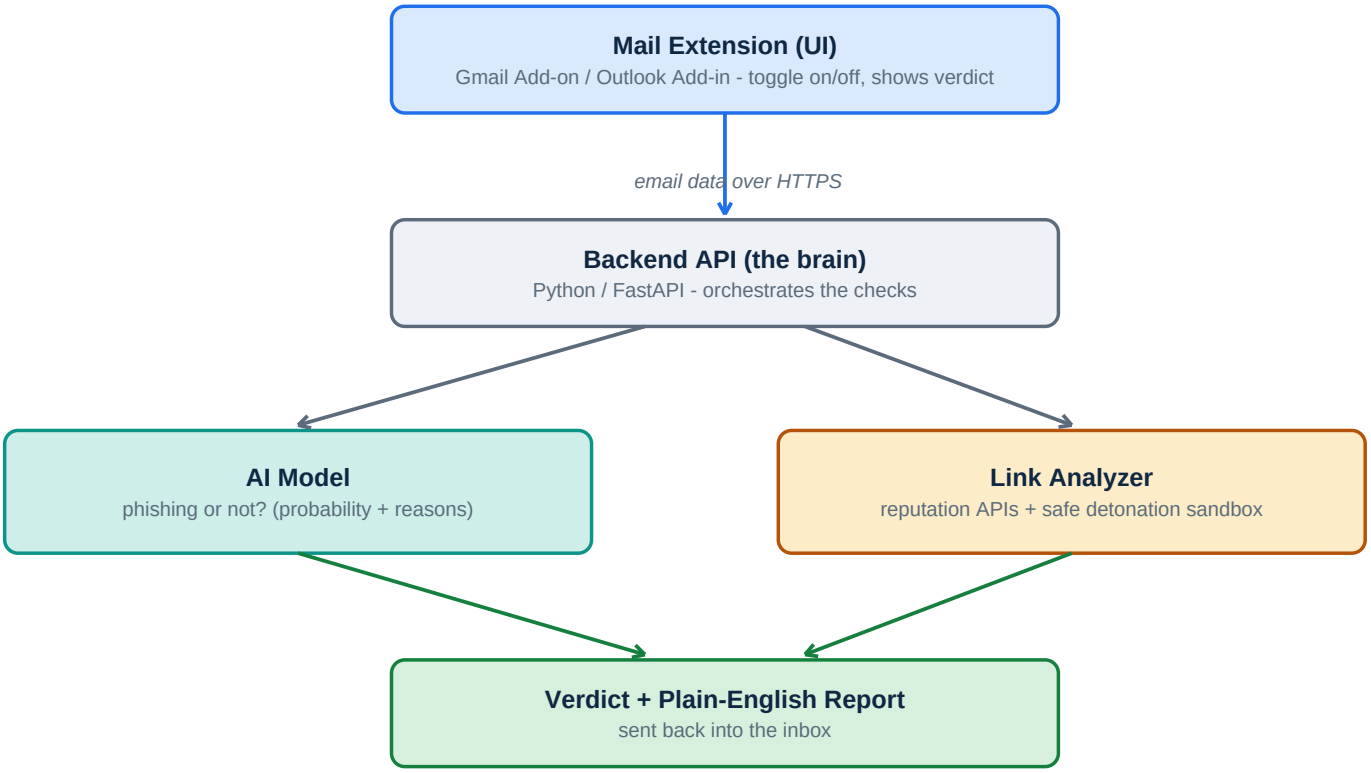

Figure 1 — How the system fits together

Author	Nitin	**Type**	Cybersecurity + AI portfolio project
Scope	Gmail + Outlook, hybrid sandbox	**Updated**	June 2026

1. The Idea in One Line

A Gmail/Outlook extension with an on/off toggle. When turned on for an email, an AI backend reads the sender, the body, and every link, decides whether the email is phishing, and — if a link looks dangerous — opens that link **in a safe sandbox (never your computer)** and reports back what the link actually does.

2. What Makes It Stand Out

Plain phishing classifiers are common. Two things make this a strong portfolio piece:

- **Safe link detonation** — you don't just guess a link is bad, you open it in an isolated environment and show the user the evidence (redirects, fake login pages, downloads).
- **Security at the human's point of decision** — right inside the inbox, on demand, with a plain-language explanation instead of a silent yes/no.

3. The Honest Reality Check

You chose the most ambitious version: **both** mail platforms, a **hybrid** sandbox, and a **full product**. That is a big build for a first serious project. The smart move is not to build it all at once. This plan is split into phases so you always have something working to show, and each phase adds one capability. A finished small thing beats an unfinished big thing — and recruiters specifically value the ability to scope and ship.

Rule of thumb: never start a phase until the previous one runs end to end.

4. System Architecture

The five building blocks:

- **The extension (frontend)** — the toggle and the result display inside Gmail/Outlook.
- **The backend API (orchestrator)** — receives the email, calls everything else, returns a verdict.
- **The AI model (classifier)** — reads body + sender + URLs, outputs a phishing probability.
- **The link analyzer** — checks each URL's reputation and, when needed, detonates it safely.
- **The reporting layer** — turns raw results into a clear, human-readable answer.

See Figure 1 on the cover for the full data flow.

5. Tech Stack (beginner-friendly)

Part	Tool	Why
Backend API	Python + FastAPI	Easiest modern Python backend; same language as your ML
AI model (start)	scikit-learn	Simple, fast, runs on a laptop, easy to explain in interviews
AI model (later)	Fine-tuned DistilBERT	Upgrade once the simple model works
Link reputation	VirusTotal, urlscan.io	Free tiers; trusted in industry
Safe detonation	urlscan.io now; Docker sandbox later	Runs in each company's own isolated VM
Gmail extension	Google Workspace Add-on (Apps Script)	No server hosting for the add-on itself
Outlook extension	Office.js add-in + Microsoft Graph	Standard Microsoft add-in path
Hosting backend	Render / Railway / Fly.io free tier	Simple deploys for beginners
Sandbox VMs (later)	Docker on a cheap cloud VM	Disposable, isolated, resettable

Check current free-tier limits and pricing for VirusTotal, urlscan.io and your host before relying on them — these change.

6. The Build, Phase by Phase

Phase 0 — Learn the moving parts

setup

about 1 week

Understand the pieces before coding.

- Collect 20–30 real phishing emails and 20–30 legit ones; see how each is structured.
- Sign up for VirusTotal and urlscan.io free API keys and read their quick-starts.
- Skim a FastAPI 'hello world' tutorial.

Deliverable: notes + working API keys.

Phase 1 — The AI model alone

CORE

2–3 weeks

Build and train the phishing classifier as a standalone program — no extension yet.

- Get data: PhishTank, Nazario corpus, Kaggle phishing sets, Enron for legit mail.
- Extract features: urgency/threat words, sender display-name vs domain mismatch, raw-IP links, look-alike domains (paypa1.com), link-text vs destination mismatch.
- Train a simple model (Logistic Regression / Gradient Boosting) → phishing probability.
- Evaluate with precision & recall (recall matters most — don't miss real attacks).
- Wrap it as predict(email) → {score, top_reasons}.

Deliverable: a script that scores an email and lists reasons. Already portfolio-worthy on its own.

Phase 2 — Backend API + link reputation

build

1–2 weeks

Turn the model into a service and add real link checks.

- Wrap the model in a FastAPI POST /analyze endpoint.
- For each link, call VirusTotal and/or urlscan.io for a reputation score.
- Combine model score + link reputation into one verdict and a list of reasons.
- Test with curl / Postman using your saved example emails.

Deliverable: a running API that returns a full verdict for an email.

Phase 3 — Safe link detonation

STANDOUT

2–3 weeks

The impressive part. Start with the safe path, then optionally go custom.

- **Step A (first):** submit suspicious links to urlscan.io — it opens them in its sandbox and returns screenshots, final URL, page content. Zero risk to any machine.
- **Step B (advanced):** your own Docker + headless-browser sandbox, destroyed and recreated after every link, with strict network isolation.
- **Deployment (your idea):** have each company run the isolated VM in their own environment — the agent just sends links to their sandbox and reads back the report (see Figure 2).

Deliverable: when a link is flagged, the user sees what is actually behind it, with evidence.

Phase 4 — Gmail extension

build

about 2 weeks

- Build a Google Workspace Add-on that shows a card with an on/off toggle while reading mail.
- When on, it sends sender/body/links to your /analyze API.
- Display verdict, reasons, and any detonation report in the Gmail sidebar.

Deliverable: working phishing check inside Gmail.

Phase 5 — Outlook extension

build

about 2 weeks

- Repeat Phase 4 with an Office.js add-in + Microsoft Graph, calling the same backend.
- Only the frontend changes — your backend is platform-agnostic.

Deliverable: the same feature inside Outlook.

Phase 6 — Full-product polish

bonus

ongoing

Pick from these to make it feel real:

- User accounts + a small dashboard (scan history, stats).
- A 'report as phishing/safe' button that improves the model over time.
- Caching, rate limiting, basic auth, a landing page + demo video.

Deliverable: a polished, demoable product for your portfolio.

Safe Detonation & Deployment

How a risky link is opened without endangering anyone's computer, and where the sandbox should live.

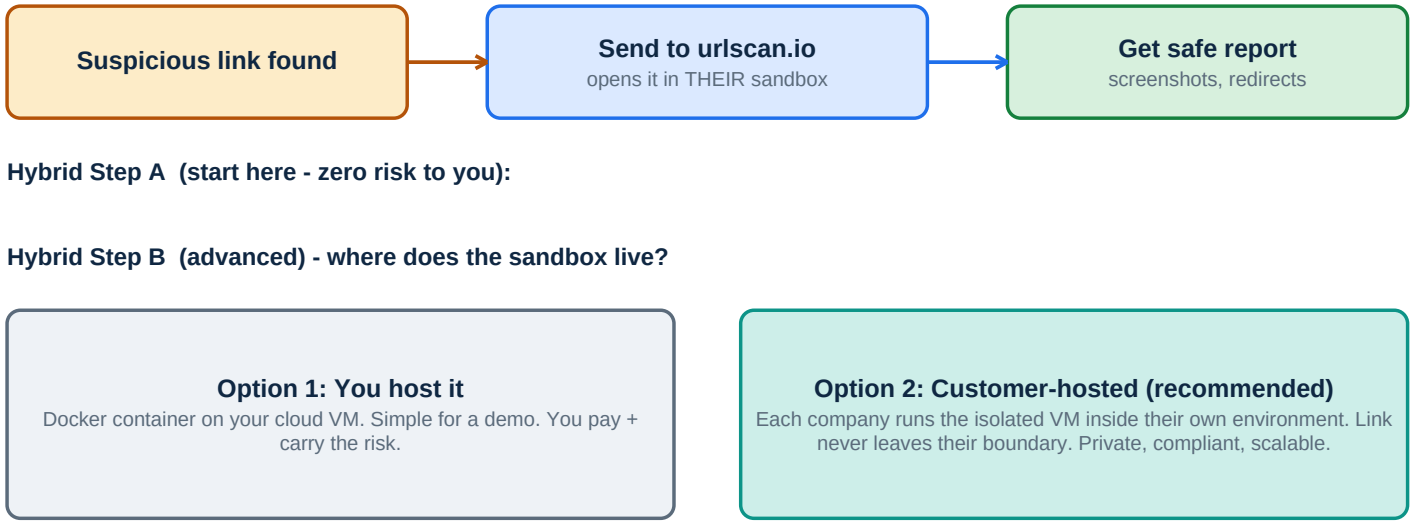

Figure 2 — Detonation flow and the two sandbox deployment models

Safety note: detonating live malicious links is genuinely risky. Until your own sandbox is fully isolated, rely on urlscan.io. Never run untrusted links on your laptop or a VM that touches your real accounts or network.

7. Suggested Timeline

Figure 3 — Rough schedule. Minimum impressive portfolio piece = Phases 1–4.

8. Skills You'll Pick Up (great for a résumé)

- Practical machine learning: feature engineering, training, evaluation (precision/recall).
- API development with FastAPI.
- Working with third-party security APIs (VirusTotal, urlscan.io).
- Containerization and sandbox isolation (Docker).
- Browser add-on development (Google Workspace + Office.js).
- Secure-by-design thinking: isolation, least privilege, never trusting input.

9. Risks & How to Handle Them

Risk	Mitigation
Detonating real malware links is dangerous	Use urlscan.io until your own sandbox is fully isolated; never your real machine
Email data is private	Don't store full emails; process and discard; be transparent about what you send
False positives (legit mail flagged)	Tune the threshold; always warn, never auto-delete; show reasons so the user decides
Scope creep / burnout	Follow the phases; ship Phase 1 before touching Phase 2
Free-tier API limits	Cache results; check current quotas before relying on them

10. Immediate Next Steps

- Set up VirusTotal + urlscan.io accounts and grab API keys.
- Collect ~50 phishing and ~50 legitimate emails into one labelled file.
- Start Phase 1: build the feature extractor and train the first simple model.

When you're ready, I can help with any of these: build the Phase 1 model step by step, write the feature-extraction code, set up the FastAPI backend, or scaffold the Gmail add-on.